Final Game Design Report

Assignment 2 – Creating a 3D PC/VR Game

Module: CPT306 – Game Design Theory and Practice

Team Members:

Haiqi Shen Jinghao Zhang

Kejie Ren Mengxuan Shen Changbo Wang

April 26, 2026

Abstract This report presents the full design and implementation of a 3D horror-themed linear escape room game developed in Unity for PC platforms, with an extensible architecture supporting future VR adaptation. The game is set in a dark, abandoned hospital environment, where players progress through three sequential levels by exploring scenes, collecting items, decoding clues, solving puzzles, and unlocking gated doors. The core gameplay follows a clear and low-friction interaction loop supported by modular functional scripts including clue inspection, item collection, password validation, door unlocking, ritual activation, and elevator escape.

The design emphasises gated progression, immediate player feedback, and state-driven system behaviour to ensure intuitive interaction and reduced frustration. The game starts with basic key-and-door mechanics, advances to clue decoding and password authentication, and culminates in a narrative-driven item collection, sacrifice ritual, and final elevator escape. Internal playtesting shows that the progression flow is logical, and players can quickly understand objectives through clear visual and textual prompts. Limitations include limited narrative depth, low environmental interactivity, and lack of puzzle variety. Future improvements will focus on atmosphere enhancement, dynamic difficulty adjustment, VR adaptation, and extended replayability.

Keywords: 3D Escape Game; Horror Environment; Puzzle Interaction; Unity Development; Player Guidance; Gated Progression

1 Introduction

1.1 Purpose and Goals

This project applies game design theories, interaction principles, and Unity development skills to design and implement a fully playable 3D horror escape room prototype. The main goals are to establish a logical three-level progression flow, design intuitive and low-friction interactions, build a modular scripting system for team collaboration, and validate the effectiveness of state-driven gated progression in a controlled horror environment.

1.2 Game Overview and Unique Hook

The game uses a first-person perspective in a confined, dimly lit abandoned hospital with strong psychological horror elements. Players advance through three linear levels by completing

sequential objectives: collecting keys, decoding clues, entering passwords, collecting ritual items, performing a sacrifice ceremony, and unlocking the elevator. The unique hook is dual-condition gated progression, where players must satisfy both item possession and knowledge requirements to unlock doors and advance, creating a tight and rewarding logical chain.

1.3 Related Work

This project references core mechanics from classic escape room and survival horror games, including environmental storytelling, lock-and-key progression, and tension-based puzzle solving. Compared with commercial titles such as *Layers of Fear* and *Resident Evil*, this prototype simplifies complex systems and focuses on deterministic linear progression, clear interaction logic, and usabilitymaking it suitable for academic validation and module assessment.

2 Game Design

2.1 Game Concept

The game is a linear 3D horror escape room experience combining exploration, clue interpretation, item collection, and sequential puzzle solving. The design prioritises clarity, low-friction operations, and strong feedback to avoid player frustration. The game is structured into three levels with increasing complexity, guiding players from basic interactions to a narrative-driven climax.

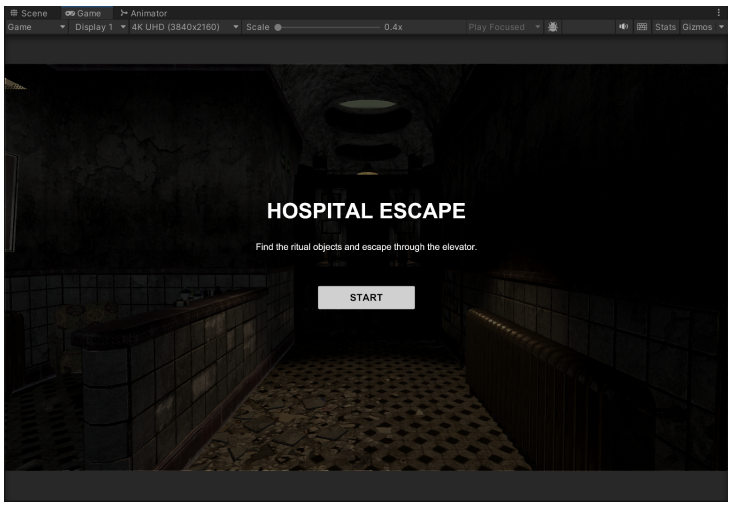


Figure 1: Main title screen and game start interface

2.2 Game World

The game world is a compact, enclosed abandoned hospital featuring wards, corridors, medical cabinets, ritual areas, and an elevator. Dark lighting, limited visibility, and spatial confinement create psychological pressure. All areas are connected by locked doors, forming a structured progression path that controls player pace and strengthens exploration purpose.

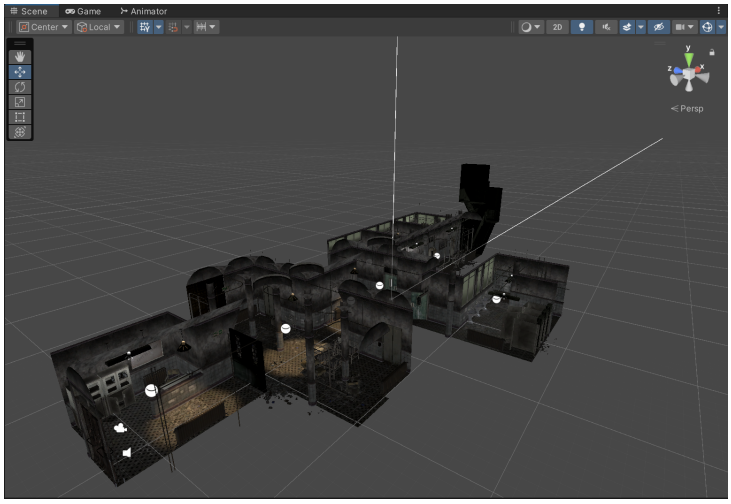


Figure 2: Overall layout and environment of the hospital escape scene

2.3 Core Game Interaction System

The game uses a unified, consistent interaction framework across all levels, including extended mechanics for the final stage:

1. **Clue Inspection**: Left-click clues to display hint information via UI panels.
2. **Item Collection**: Click keys or ritual items to update global states and remove objects from the scene.
3. **Password Authentication**: Input four-digit passwords to unlock encrypted doors.
4. **Door Unlocking**: Press E near trigger zones to open doors when conditions are satisfied.
5. **Ritual Activation**: Collect all required items and press E at the ritual site to complete the ceremony.
6. **Elevator Escape**: Use the unlocked elevator to trigger final level completion.

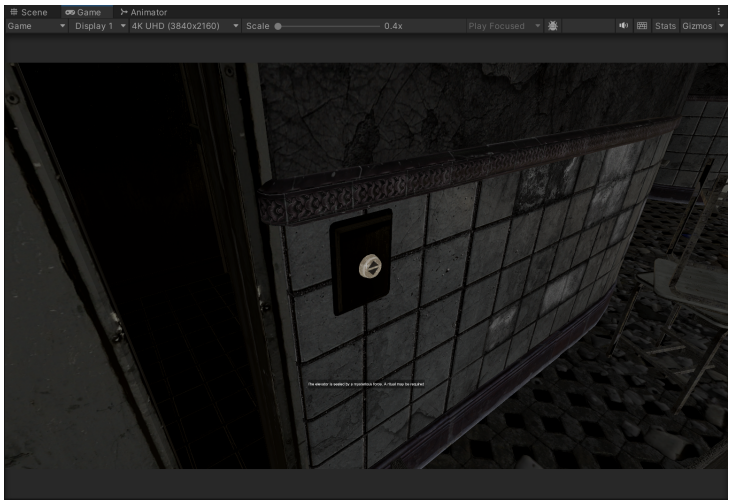


Figure 3: Context-sensitive interaction feedback with elevator control panel

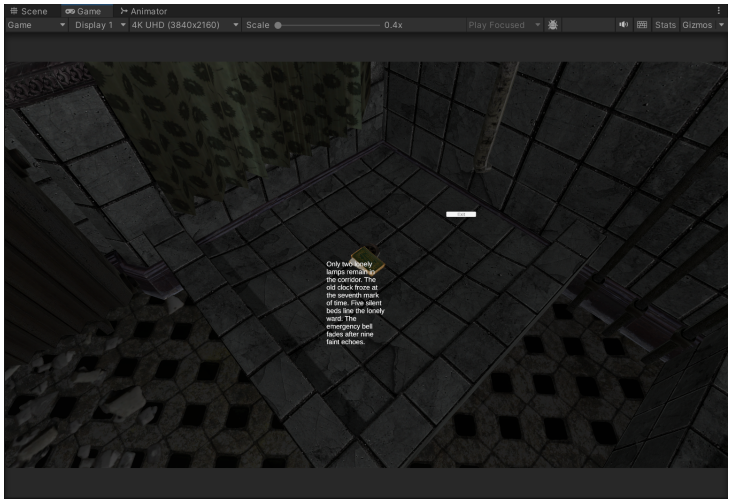


Figure 4: Clue interaction panel displaying narrative hint information

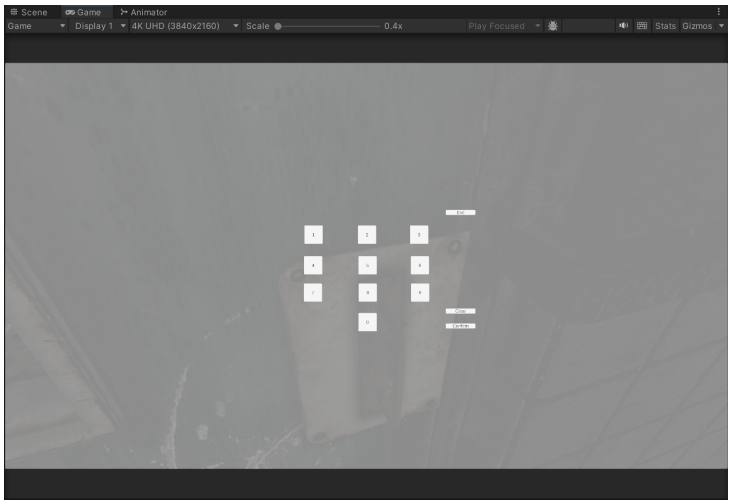


Figure 5: Password input UI panel with digit validation system

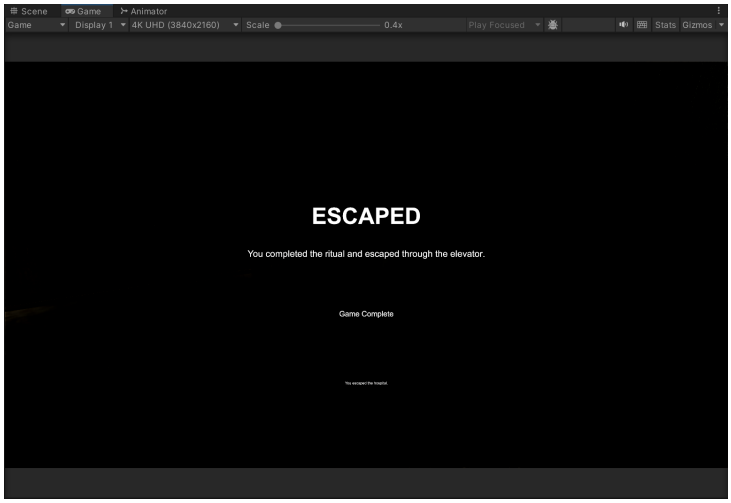


Figure 6: Escaped

2.4 Gameplay Mechanisms and Design Theory

The prototype applies evidence-based game design principles across all puzzle and interaction systems:

- **Gated Progression**: Doors, rituals, and elevators act as hard gates to ensure ordered progression.
- **Feedback Loop Design**: Every action triggers visual, textual, or animated feedback.
- **State-Driven System**: Boolean variables and counters form a finite state machine for stable behaviour.
- **Error-Tolerant Interaction**: Input restrictions and UI protection reduce player frustration.
- **Progressive Complexity**: Mechanics increase gently from key collection to multi-item ritual completion.

2.5 Full Three-Level Progression Design

The game consists of three linearly connected levels with increasing mechanical and narrative depth, sharing the same interaction system.

2.5.1 Level 1: Key Collection and Basic Door Unlocking

Level 1 acts as a tutorial. Players explore the starting area, locate and collect a hidden key, then use it to open the first door. This level teaches basic movement, item collection, and door interaction.

2.5.2 Level 2: Clue Decoding and Password Door Escape

Level 2 introduces clue interpretation. Players find a notebook, deduce a four-digit password, and input it into a password panel to unlock the next area.

2.5.3 Level 3: Ritual Completion and Elevator Escape

Level 3 is the final narrative-driven stage. Players explore the deeper hospital to collect three hidden ritual items, complete a sacrifice ritual at the designated circle, unlock the sealed elevator, and enter it to achieve full escape.

Three ritual items are distributed across the scene to encourage full exploration.

Item Name	Placement Location	Function
Ritual Item 1	Ward area	Advance ritual progress
Ritual Item 2	Near medicine cabinet	Advance ritual progress
Ritual Item 3	Corridor or hidden corner	Advance ritual progress

Table 1: Key Item Design for Level 3

The system tracks collection count in real time. When players approach the ritual site:

- Incomplete collection: on-screen hint reminds players to continue searching.
- Full collection: players can press E to complete the ritual and unlock the elevator.

The elevator remains locked until the ritual is finished. The final escape follows a strict logical condition chain:

$$CollectItems = 3 \Rightarrow RitualCompleted = true \Rightarrow ElevatorUnlocked = true \Rightarrow GameComplete$$

All mechanics are implemented using Unity engine components and custom C# scripts.

Technology/Tool	Purpose
Unity	Scene construction, physics, animation, runtime
C#	Interaction, collection, ritual, and escape logic
Collider Trigger	Detect ritual, elevator, and exit zones
Animator	Elevator and ritual animations
Canvas UI	Interface, messages, and completion screen
TextMesh Pro	Text and button rendering
Post Processing	Visual atmosphere and horror effects

Table 2: Technology Stack for Core Mechanics

3 Implementation

3.1 Team Responsibilities

Haiqi Shen developed core gameplay scripts including Door.cs, DoorPassword.cs, Key.cs, ClueInteract.cs, ritual and elevator logic. Jinghao Zhang and Kejie Ren designed levels, constructed 3D scenes, integrated art assets, and built environmental atmosphere. Mengxuan Shen and Changbo Wang implemented the UI system, conducted internal playtesting, recorded usability issues, and contributed to report writing.

3.2 Development Steps and Tools

The project uses Unity 2022.3 LTS and C#. Additional tools include Blender for model editing, Photoshop for UI textures, and Unity Asset Store for environment resources. AI tools including GPT-4o and GitHub Copilot assisted with script optimisation, debugging, and framework design. The development pipeline includes puzzle design, script implementation, UI integration, animation, and iterative testing.

3.3 Implementation Issues and Solutions

Three key technical issues were resolved:

1. UI clicks conflicting with world raycasts: fixed by adding UI raycast detection.
2. Inconsistent data types causing input errors: solved by unified validation logic.
3. Unclear feedback during repeated interactions: improved with timed state-based hints.

3.4 Preliminary Player Experience Analysis

Six internal testers participated. All players mastered key-door logic within 10 seconds. Password interaction was quickly learned, though clue-theme mapping could be strengthened. Some small objects were initially hard to notice. Average completion time ranged from 3 to 5 minutes, indicating appropriate difficulty.

4 User Manual

4.1 How to Run

Open the project in Unity 2022.3 LTS or newer, load the main gameplay scene, and click Play in the editor or build and run the exported PC executable.

4.2 Basic Controls

WASD for movement; mouse for camera rotation; left-click to inspect clues or collect items; E to open doors, activate rituals, or use elevators; UI buttons for password input and confirmation.

5 Discussion and Conclusion

5.1 Critical Discussion

The prototype delivers a complete, modular, and playable escape room experience with clear progression, consistent interactions, and effective dual-condition gating. Feedback systems significantly reduce confusion. However, limitations remain: shallow narrative, limited environmental interaction, minimal puzzle variety, weak visual guidance for interactive objects, and fully linear progression. Ethically, the game uses mild psychological horror through lighting and environment without violence, blood, or jump scares, making it suitable for a general adult audience.

5.2 Future Work

Future improvements include expanding level content, adding diverse puzzle types, enhancing horror atmosphere with sound and dynamic events, implementing highlight effects for interactive objects, introducing dynamic difficulty adjustment, supporting VR input, and adding narrative branches to improve replayability.

5.3 Conclusion

This project presents a complete 3D horror escape room prototype from design to implementation. It successfully applies gated progression, feedback-loop design, and state-driven interaction in a Unity environment. The modular architecture and clear three-level flow provide a solid foundation for extension and migration. The project fully meets all requirements of CPT306 Assignment 2 and demonstrates effective team collaboration and professional game design practice.

Statement of Academic Honesty

By submitting this coursework for assessment, we confirm that we have read and understood the Universitys policy on plagiarism and collusion. This work is created solely by the group members listed above. All use of AI tools and external resources is appropriately declared and referenced. No part of this report is copied from uncredited sources.

References

1. Unity Technologies. *Unity Manual and Scripting API.* `https://docs.unity3d.com`
2. Fullerton, T. *Game Design Workshop.* CRC Press, 2019.
3. Schell, J. *The Art of Game Design: A Book of Lenses.* CRC Press, 2020.
4. OpenAI. (2026). GPT-4o Large Language Model [AI software].
5. Microsoft. (2026). GitHub Copilot [AI programming assistant].